TECHNIQUESFOR BINARYTREES

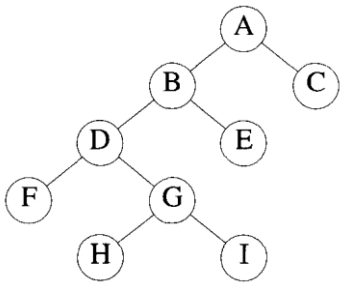
The solution to many problems involves the manipulation of binary trees, trees, or graphs. For example, we may wish to find all vertices in a binary tree with a data value less than x or we may wish to find all vertices in a given graph G that can be reached from another given vertex v. The determination
of this subset of vertices satisfying a given property can be carried out by systematically examining the vertices of the given data object. This often takes the form of a search in the data object involves the examination of every vertex in the object being searched, it is called a traversal.

There are many operations that we want to perform on binary trees. If we let L,D,and R stand for moving left, printing the data, and moving right when at a node, then there are six possible combinations of traversal: LDR, LRD, DLR, DRL, RDL, and RLD. If we adopt the convention that
we traverse left before right, then only three traversals remain: LDR, LRD, and DLR. To these we assign the names inorder , postorder , and preorder.

```
treenode= record
{
Type data;
// Type is the datatype of data,
treenode*lchildtr;eenode *rchild;
}
 AlgorithmInOrder(i)
// t is a binary tree.Eachnodeof t has
//
threefields:Ichildd,ata,and rchild.
{
if t!= 0 then
{
 In0rder(i-> Ichild);
Visit(i);
In0rder(i-> rchild);
 }
}

 AlgorithmPreOrder(i)
// t is a binary tree.Each node of t has
 // three fields:Ichild data,and rchild.
 {
 if t!= o then
 {
 Visit(i);
 PreOrder(i -> Ichild);
 PreOrder(i -> rchild);
 }
 }

 AlgorithmPostOrder(i)
// t is a binary tree.Each node of t has
 //
threefields:Ichildd,ata,and rchild.
 {
 if t!= 0 then
{
PostOrder(i -> Ichild);
PostOrder(i-> rchild);
 Visit(i);
} }
Example
```



call of InOrder	value in root	action
main	A	
1	B	
2	D	
3	F	
4	—	print ('F')
4	—	print ('D')
3	G	
4	H	
5	—	print ('H')
5	—	print ('G')
4	I	
5	—	print ('I')
5	—	print ('B')
2	E	
3	—	print ('E')
3	—	print ('A')
1	C	
2	—	print ('C')
2	—	

TECHNIQUES FOR GRAPHS

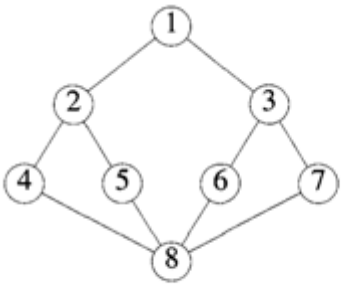
A fundamental problem concerning graphs is the reach ability problem. In its simplest form it requires us to determine whether there exists a path in the given graph G = (V, E) such that this path starts at vertex v and ends at vertex u. A more general form is to determine for a given starting vertex v ∈V all vertices u such that there is a path from v to u. This latter problem can be solved by starting at vertex v and systematically searching the graph G for vertices that can be reached from v

Breadth First Search and Traversal

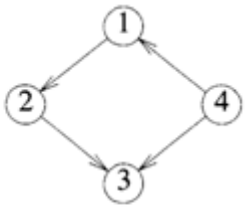
In breadth first search we start at a vertex v and mark it as having been reached. The vertex v is at this time said to be un explored. A vertex is said to have been explored by an algorithm when the algorithm has visited all vertices adjacent from it. All unvisited vertices adjacent from v are visited next. These are new unexplored vertices. Vertex v has now been explored. The newly visited vertices haven't been explored and are put onto the end of a list of unexplored vertices. The first vertex on this list is the next to be explored. Exploration continues until no unexplored vertex is left.

```
// Breadth-First Search of G starting at vertex v
{
 u := v;
```

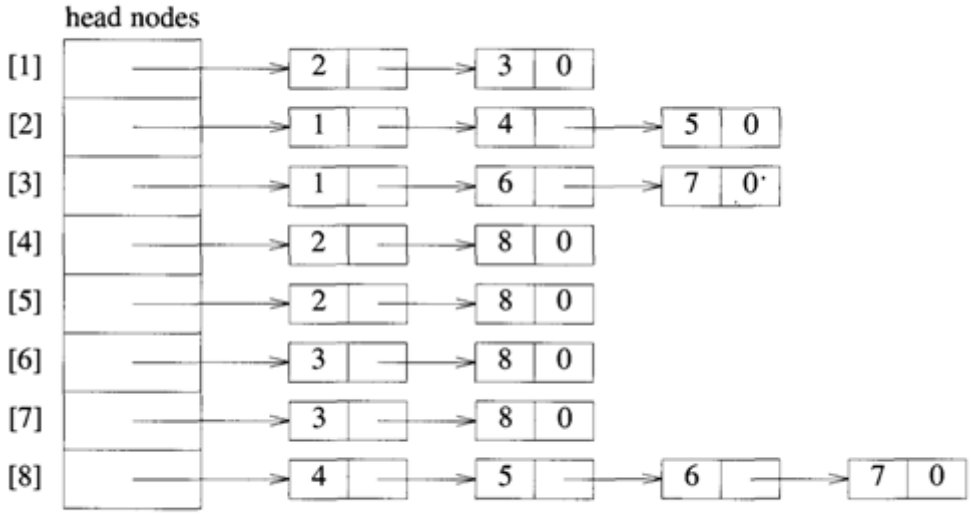
```
  visited[v] := 1;
  repeat
    for all vertices w adjacent to u do
      if (visited[w] = 0) then
        Add w to q;
        visited[w] := 1;
      end if
    end for
    if q is empty then return; // No unexplored vertex
    Delete u from q; // Get first unexplored vertex
  until false;
}
```



(a) Undirected graph G



(b) Directed graph



(c) Adjacency list for G

```
Algorithm BFT(G, n)
{
  for i := 1 to n do
    visited[i] := 0; // Mark all vertices unvisited
  for i := 1 to n do
```

```
   if (visited[i] = 0) then BFS(i);
}
```

Depth First Search and Travers

A depth first search of a graph differs from a breadth first search in that the exploration of a vertex v is suspended as soon as a new vertex is reached. At has been explored, the exploration of v continues. The search terminates when all reached vertices have been fully explored

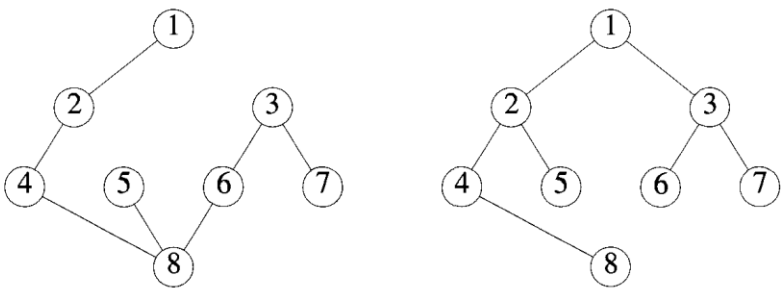
```
Algorithm DFS(w)
{
 visited[v] := 1;
 for each vertex w adjacent to v do
   if (visited[w] = 0) then DFS(w);
}
```

CONNECTED COMPONENTS AND SPANNING TREES

If G is a connected undirected graph, then all vertices of G will get visited on the first call to BFS. If G is not connected, then at least two calls to BFS will be needed. Hence, BFS can be used to determine whether G is connected. Furthermore, all newly visited vertices on a call to BFS from BFS represent the vertices in a connected component of G. Hence, the connected components of a graph can be obtained using BFS. For this, BFS can be modified so that all newly visited vertices are put onto a list. Then the subgraph formed by the vertices on this list makes up a connected component. Hence, if adjacency lists are used, a breadth-first traversal will obtain the connected components in O(n + e) time.

As a final application of breadth-first search, consider the problem of obtaining a spanning tree for an undirected graph G. The graph G has a spanning tree if and only if G is connected. Hence, BFS easily determines the existence of a spanning tree

Furthermore, consider the set of edges (u, w) used in the for loop of line 11 of algorithm BFS to reach unvisited vertices w. These edges are called forward edges. Let t denote the set of these forward edges. We claim that if G is connected, then t is a spanning tree of G. For the graph of below Figure (a), the set of edges t will be all edges in G except (5, 8), (6, 8), and (7, 8) (see below Figure b). Spanning trees obtained using a breadth-first search are called breadth-first spanning trees



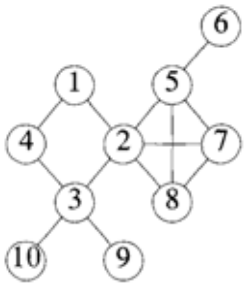
(a) DFS(1) spanning tree (b) BFS(1) spanning tree

BICONNECTEDCOMPONENTSAND DFS

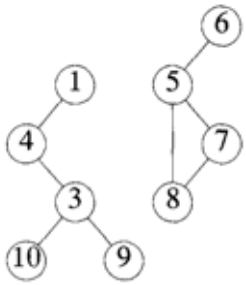
"In this section, by 'graph' we always mean an undirected graph. A vertex v in a connected graph G is an articulation point if and only if the deletion of vertex v, together with all edges incident to v, disconnects the graph into two or more nonempty components."

The graph of Figure a ,b is not biconnected. The graph of Figure c below is biconnected. The presence of articulation points in a connected graph can be an undesirable feature in many cases.

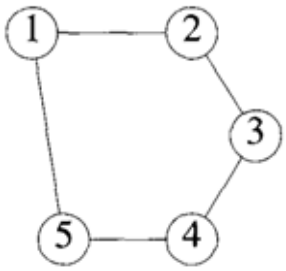
In this section, we develop an efficient algorithm to test whether a connected graph is biconnected. For the case of graphs that are not biconnected, this algorithm will identify all the articulation points. Once it has been determined that a connected graph G is not biconnected, it may be desirable to determine a set of edges whose inclusion makes the graph biconnected



(a) Graph *G*



(b) Result of deleting vertex 2



C

)

For example, if G represents a communication network with the vertices representing communication stations and the edges communication lines, then the failure of a communication station i that is an articulation point would result in the loss of communication to points other than i too. On the other hand, if G has no articulation point, then if any station i fails, we can still communicate between every two stations not including station i.

we develop an efficient algorithm to test whether a connected graph is biconnected. For the case of graphs that are not biconnected, this algorithm will identify all the articulation points. Once it has been determined that a connected graph G is not biconnected, it may be desirable to determine a set of edges whose inclusion makes the graph biconnected. Determining such a set of edges is facilitated if we know the maximal subgraphs of G that are biconnected. G' = (V', E') is a maximal biconnected subgraph of G if and only if G has no biconnected subgraph G" = (V", E"). Such that V ⊆ V" and E' ⊆ E". A maximal biconnected subgraph is a biconnected component.

Example :Using the above scheme to transform the graph of Figure (a) into a biconnected graph requires us to add edges (4, 10) and (10, 9) (corresponding to the articulation point 3), edge (1, 5) (corresponding to the articulation point 2), and edge (6, 7) (corresponding to the articulation point 5).

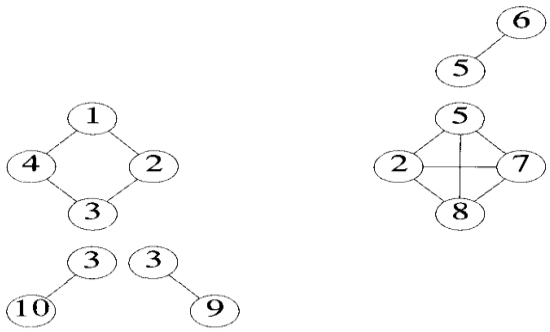
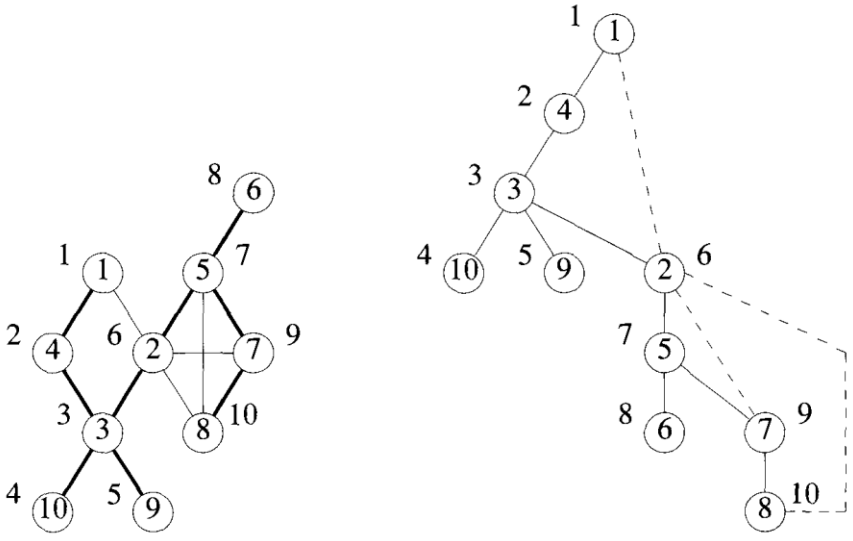


figure (d)

for each articulation point a do
{
Let B1, B2, ..., Bk be the biconnected components containing vertex a;
Let Vi, Vj be a vertex in Bi, 1 < i < k;
Add to G the edges (wi, wi+1), 1 < i < k;
}

Figure (e) and (f) show a depth-first spanning tree of the graph of Figure a. In each figure, there is a number outside each vertex. These numbers correspond to the order in which a depth-first search visits these vertices and are referred to as the depth-first numbers (dfns) of the vertex. Thus, dfn[1] = 1, dfn[4] = 2, dfn[6] = 8, and so on. In Figure 6.9(b), solid edges form the depth-first spanning tree. These edges are called tree edges. Broken edges (i.e., all the remaining edges) are called back edges.



Algorithm BiComp(u, v)

```
// u is a start vertex for depth-first search, v is its parent if any in the depth-first spanning
//tree. It is assumed that the global array dfn is initially zero and that the global variable
//num is initialized to 1. n is the number of vertices in G.
dfn[u] := num; L[u] := num; num := num + 1;
for each vertex w adjacent from u do
{
   if ((w != v) and (dfn[w] < dfn[u])) then
      add (u, w) to the top of a stack;
   if (dfn[w] = 0) then
   {
      if (L[w] > dfn[u]) then
      {
         write ("New biconnected component");
         repeat
         {
            Delete an edge from the top of stack;
            Let this edge be (x, y);
            write (x, y);
         } until ((x, y) = (u, w)) or ((x, y) = (w, u));
      }
      BiComp(w, u);  // w is unvisited.
      L[u] := min(L[u], L[w]);
   }
   else if (w != v) then
      L[u] := min(L[u], dfn[w]);
}
}
}
```